WEBSITE IMPROVEMENT TECHNICAL REPORT

A practical proposal for improving Cynbit’s project inquiry experience

Prepared by	Priyansh Jain
Role	Full Stack Developer Intern — Technical Assessment
Company website reviewed	cynbit.in
Focus area	Project inquiry form and lead qualification
Report date	20 October 2026

Proposal summary

Cynbit’s contact page already gives visitors a direct way to send an inquiry. The proposed improvement is a guided project brief that helps visitors describe what they need with less uncertainty, while giving the team a more structured first message.

This document describes the observed opportunity, recommended interaction, technical approach, acceptance criteria, and a realistic validation plan. It is a technical proposal—not a claim that code has already been deployed or that conversion results have been measured.

Prepared for the Full Stack Developer Intern assessment

1. Problem statement

Cynbit's public Contact page invites prospective clients to share a name, email, optional company and industry, and a message. That is a sensible starting point, but a single open-ended message can leave important project details unclear. A visitor may not know what information to include, and the team may need a follow-up just to understand the request.

The improvement

Add a short, responsive "Project Brief" flow beside or in place of the free-form inquiry form. It should ask only for details that help route and understand an inquiry, and it should still allow a visitor to submit a simple message without completing every optional field.

User and developer perspectives

Perspective	Current friction / opportunity	Proposed response
Visitor	A blank message field gives little guidance about what to write.	Use clear prompts, examples, and a visible step indicator.
Visitor	Visitors may not know what information matters for a first conversation.	Ask for service type, short goal, timeframe, and optional budget range.
Cynbit team	Requests can arrive with inconsistent levels of detail.	Create a structured summary that is easy to scan and route.
Developer	Form quality depends on validation and reliable handling.	Validate on the client and server; provide clear success/error states.

Success criteria

- A first-time visitor can understand the form without instructions from a staff member.
- The flow works on mobile and desktop, with keyboard-accessible controls and clear labels.
- Required fields are checked and errors are shown beside the relevant input.
- The user can review the project summary before submission.
- The form does not imply that an inquiry has been sent until the server confirms success.

2. Proposed user experience

The change is intentionally small. It improves the existing contact journey instead of introducing an unrelated feature or forcing visitors through a long questionnaire.

Suggested flow

01 — Contact details

Name and email are required. Company is optional.

02 — Project context

Choose a service category such as web/app development, AI automation, cloud, or digital marketing.

03 — Project brief

Describe the goal in a few sentences. Add optional timeframe and budget range if known.

04 — Review and send

Show a compact summary, allow edits, then submit with a clear loading and success state.

Interaction details

• **Progress:** show the current step and how many remain; do not use a progress bar as the only status cue.

• **Validation:** validate required fields on submission and preserve entered values when errors occur.

• **Optionality:** budget and timeframe should remain optional to avoid discouraging early-stage inquiries.

• **Accessibility:** visible labels, sensible tab order, focus styles, readable contrast, and text-based error messages.

• **Responsive layout:** stack fields on narrow screens and keep the primary action easy to reach.

• **Privacy:** collect only information needed to respond; link the existing privacy notice where available.

Example project brief

Service: AI automation Timeline: 1–3 months
Goal: “We want to reduce repetitive document processing and route requests automatically.”
Budget: Not decided

This is an illustrative example, not a real customer inquiry or measured user test.

3. Technical approach

The prototype can be built with a small, maintainable full-stack structure. The final stack should follow the conventions already used in the live Cynbit codebase if access to that repository is provided.

Layer	Recommendation	Reason
Frontend	React / Next.js with accessible form components and responsive CSS	The Full Stack Developer role references React and Next.js; the flow benefits from component state.
Backend	Node.js API route or existing backend endpoint	Validates input, normalizes values, and owns submission status.
Validation	Client-side checks plus server-side schema validation	Client validation improves feedback; server validation is the trust boundary.
Data handling	Existing CRM/email integration or a minimal lead record in the current datastore	Avoid creating a parallel lead database if an existing workflow is available.
Quality	Unit tests for validation and an end-to-end happy/error path	Protects required fields, errors, and successful submission behavior.

Example request contract

```
POST /api/inquiries Content-Type: application/json { "name": "Required string", "email": "Valid email address", "company": "Optional string", "service": "Selected category", "goal": "Short project description", "timeline": "Optional range", "budget": "Optional range" }
```

Backend responsibilities

• Reject malformed requests and validate every field again on the server.

• Normalize whitespace and enforce maximum lengths to reduce invalid or abusive input.

• Use environment variables for credentials and third-party integration settings; never commit secrets.

• Return predictable HTTP responses (for example, 201 on success, 400 for invalid input, and 500 for unexpected failures).

• Add rate limiting and spam protection before exposing the endpoint publicly.

4. Implementation plan and verification

A safe implementation should be delivered in small steps so the existing contact page continues to work while the new experience is tested.

Implementation sequence

1. Inspect

Confirm the current frontend framework, contact endpoint, privacy notice, and any existing CRM/email handling.

2. Build

Create reusable step components, form state, field validation, and a review screen.

3. Connect

Submit the validated payload to the existing backend workflow; add server-side validation and useful error responses.

4. Test

Cover valid submission, missing name/email, invalid email, long input, server failure, keyboard navigation, and mobile viewport.

5. Deploy

Use a feature branch, review the diff, run tests/build, deploy to a preview environment, then publish the production change after verification.

Test checklist

Test	Expected result
Required fields empty	Inline messages appear; no request is sent.
Invalid email	User is asked to correct the email address.
Valid project brief	Review summary matches the entered values.
Backend returns success	Confirmation is displayed only after success.
Backend fails / times out	An understandable error appears; values remain available to retry.
Mobile width and keyboard-only use	No horizontal overflow; all controls can be reached and operated.

5. Results and measurement

No performance or conversion results are claimed at proposal stage. After release, compare the guided form with the existing form using a defined measurement window and enough traffic to interpret the results responsibly.

Primary metric: completed valid inquiries divided by inquiry-form starts.
Supporting metrics: form abandonment by step, validation error rate, submission failure rate, and completeness of the submitted project brief.
Guardrails: page performance, accessibility regressions, spam volume, and privacy compliance.

The change should be retained only if it makes inquiries easier to submit and improves the usefulness of the information received without materially increasing abandonment.

6. GitHub, deployment, and submission checklist

Suggested repository layout

```
src/ components/ ProjectInquiryForm/ ProjectInquiryForm.jsx FormStep.jsx ReviewSummary.jsx
lib/ inquiryValidation.js pages-or-app/ api/inquiries.js tests/ inquiryValidation.test.js
inquiry-flow.spec.js README.md
```

Example clean commit sequence

- `feat: add guided project inquiry steps`
- `feat: validate and submit project inquiry`
- `test: cover inquiry validation and error states`
- `docs: add setup and deployment instructions`

Before submitting

- Publish the source repository and ensure the README includes setup, environment variables, test commands, and screenshots.
- Deploy the prototype to a public URL and test it in an incognito browser and on a mobile device.
- Replace this report's proposal-stage wording with verified implementation details and actual links.
- Do not claim that a deployment, test, or metric exists unless it has been completed and checked.
- Submit the repository URL, deployed prototype URL, and final report through the assessment portal.

Reference pages reviewed

Cynbit Technologies — Contact: https://www.cynbit.in/contact
Cynbit Technologies — Careers: https://www.cynbit.in/careers
Full Stack Developer Intern role listing: LinkedIn role description

Closing note

This proposal focuses on a specific user journey and a practical full-stack implementation. The next step is to build and deploy the prototype, validate the end-to-end behavior, and update the report with the verified repository and live-demo links.